

Church App

Documentation du projet

Church App — Centre International des Vainqueurs

Objectif

Application web pour la communauté du Centre International des Vainqueurs (CIV) : suivi des sessions de prière (live/rediffusion), intentions de prière, pensée du jour, vidéothèque (cultes, veillées, camps, retraites), et un espace admin pour les responsables (admin/pasteur).

Stack technique

- **Next.js 16** (App Router, Turbopack). Cette version renomme `middleware.ts` en `proxy.ts` (export proxy au lieu de middleware) — voir [src/proxy.ts](#). Toujours vérifier `node_modules/next/dist/docs` avant d'utiliser une API Next, certaines conventions ont changé.
- **React 19.2.4**
- **Supabase** ([@supabase/ssr](#), [@supabase/supabase-js](#)) — auth, base Postgres, Storage, Realtime (notifications).
- **Tailwind CSS v4** (via [@tailwindcss/postcss](#), classes "canonical" type `bg-linear-to-r` au lieu de `bg-gradient-to-r`).
- **lucide-react** pour les icônes.

Rôles utilisateurs

UserRole = 'fidele' | 'pasteur' | 'admin' (voir [src/types/index.ts](#))

- **fidele** : accès au dashboard, prières, vidéothèque, intentions personnelles.
- **pasteur / admin** : accès en plus à l'espace /admin (vue d'ensemble avec statistiques/graphes, publication de prières/vidéos/pensées, gestion des utilisateurs — changer de rôle, désactiver/réactiver, ou **supprimer définitivement** un compte).

La désactivation (`est_actif`) bloque l'accès sans perdre les données et passe par le client normal + RLS. La suppression définitive est irréversible, supprime aussi le compte d'authentification, et passe par une route serveur dédiée avec la clé `service_role` (voir [BACKEND.md](#)).

Structure des routes

<code>src/app/</code>	
<code>(auth)/login, (auth)/register</code>	– pages publiques d'authentification
<code>(auth)/mot-de-passe-oublie</code>	– demande de réinitialisation (email)
<code>(auth)/reinitialiser-mot-de-passe</code>	– saisie du nouveau mot de passe
<code>auth/confirm</code>	– route serveur (pas dans (auth)) qui
<code>valide les liens reçus par email</code>	
<code>(dashboard)/dashboard</code>	– accueil fidèle (pensée du jour, live, sessions à
<code>venir)</code>	
<code>(dashboard)/prieres, /prieres/[id]</code>	– sessions de prière
<code>(dashboard)/videos</code>	– vidéothèque publique
<code>(dashboard)/dashboard/intentions</code>	– intentions de prière du fidèle
<code>(dashboard)/admin/*</code>	– espace admin/pasteur (vue d'ensemble,
<code>prières, vidéos, pensées, utilisateurs)</code>	

Flux d'authentification

1. `register/page.tsx` — `supabase.auth.signUp()` puis insertion immédiate d'une ligne dans utilisateurs (rôle par défaut fidele). Sans cette insertion, l'utilisateur reste "orphelin" (pas de profil) — voir [BACKEND.md](#) pour l'historique du bug correspondant.
2. `login/page.tsx` — `signInWithPassword()` puis lecture du rôle pour rediriger vers /admin ou /dashboard. Lien "Mot de passe oublié ?" vers /mot-de-passe-oublie.
3. **Mot de passe oublié** : `mot-de-passe-oublie/page.tsx` appelle `resetPasswordForEmail()` avec `redirectTo` pointant vers `/auth/confirm?next=/reinitialiser-mot-de-passe`. La route serveur `src/app/auth/confirm/route.ts` vérifie le `token_hash` reçu (`verifyOtp`) et établit la session via cookies avant de rediriger vers `/reinitialiser-mot-de-passe`, qui appelle `updateUser({ password })`. Nécessite d'avoir modifié le template d'email "Reset Password" dans le Dashboard Supabase — voir [BACKEND.md](#) pour le détail et le piège rencontré (`@supabase/ssr` force le flow PKCE, incompatible avec le format de lien par défaut de Supabase).
4. `src/proxy.ts` (équivalent middleware) — sur chaque requête : - protège /dashboard, /admin, /prieres (redirige vers /login si pas connecté, mais ignore les erreurs réseau transitoires pour éviter les redirections en boucle) - si

connecté et sur /login, redirige selon le rôle.

5. `src/lib/supabase/auth.ts` — `getAuthUser()` / `getProfil()` (cache React), utilisés par les layouts serveur pour le contrôle d'accès final.

Design

Composants UI réutilisables dans `src/components/ui/` (Button, Input, Textarea, StatCard, Donut, BarList). Halos dégradés flous (blur-3xl) utilisés en arrière-plan sur les layouts (auth) et (dashboard) pour donner du relief. Les deux navbars (principale et secondaire admin) surlignent l'onglet actif via `usePathname`.

Apparence (sombre fixe) et langue (FR/EN)

- **Apparence** : un seul thème, sombre, fixe — pas de sélecteur. Testé puis retiré le 2026-06-26 : un système clair/sombre/auto avait été mis en place (ThemeContext, classe `.dark` sur `<html>`, `@custom-variant dark` dans `globals.css`), mais l'utilisateur a préféré revenir à l'identité visuelle d'origine (noir dégradé de bleu, sans le motif de points en arrière-plan). ThemeContext.tsx a été supprimé ; le mécanisme CSS `.dark/@custom-variant` reste en place dans `globals.css` (classe `dark` appliquée en dur sur `<html>` dans `layout.tsx`) car de nombreux composants utilisent encore des paires `<classe> dark:<classe>` — les garder est sans risque tant que `dark` est toujours présent.
- **Langue** : dictionnaire simple (pas de librairie i18n) dans `src/lib/i18n/translations.ts`, choix fr/en persisté dans `localStorage` (locale), hook `useLocale()` → `t(cle)`. Exposé via la page **Paramètres** (/dashboard/parametres, accessible à tous les rôles, lien icône dans la navbar principale) — qui ne contient plus que ce réglage.
- **Portée de la traduction (progressive)** : navbars, page Paramètres, et le contenu de (auth)/login/(auth)/register/(dashboard)/dashboard. Le reste (admin/*, prières, vidéothèque, intentions...) reste en français non traduit — à étendre au fil des prochaines demandes.

La pensée du jour est affichée en bandeau recadré (object-cover) sur le tableau de bord ; cliquer dessus ouvre une lightbox ([CartePenseeDuJour.tsx](#)) qui montre l'image entière (object-contain), sans recadrage.

La vue d'ensemble admin combine StatCard (compteurs), BarList (comparaison du contenu publié : vidéos/pensées/lives/sessions planifiées) et Donut (pourcentages : fidèles actifs, fidèles ayant déjà participé à un live), plus un bandeau dédié au live en cours avec le nombre de participants connectés.

Sessions de prière en direct

/admin/prieres — bouton "Démarrer un live" ouvre [DemarrerLiveModal.tsx](#) (titre, lien YouTube Live optionnel, description) plutôt que le formulaire générique. La diffusion vidéo réelle passe par YouTube Live (l'admin diffuse depuis son téléphone ou OBS) — **diffuser une webcam in-app vers plusieurs fidèles nécessiterait un serveur de streaming externe (SFU/RTMP→HLS), hors de portée de Next.js/Supabase seuls**, voir l'historique dans [BACKEND.md](#).

Pendant le live (statut=en_cours), [LiveInteractions.tsx](#) s'affiche côté admin et côté fidèle (dashboard + page détail prière) : nombre de connectés (Presence Realtime, sans table SQL), likes et commentaires (tables `live_likes/live_commentaires`). Le bouton "Terminer le live" (icône carrée, visible uniquement sur une session en_cours) passe la session en statut=termine et type=rediffusion — elle apparaît alors automatiquement dans /prieres (filtré sur statut=termine) pour tous les fidèles.

Documentation API (Swagger)

[public/openapi.yaml](#) documente le backend du projet — qui est **Supabase** lui-même (pas de serveur API maison) : chaque table est documentée comme un endpoint REST (conventions PostgREST), avec ses politiques RLS rappelées dans la description de chaque route, plus la route serveur Next.js DELETE `/api/admin/utilisateurs/{id}`.

Pour consulter/tester : lancer l'app (`npm run dev`) puis ouvrir `http://localhost:3000/api-docs.html` ([public/api-docs.html](#), Swagger UI via CDN, pas de dépendance npm ajoutée). Lien aussi présent en bas de la page /admin (Vue d'ensemble). En production, accessible au même chemin sur le domaine déployé — partageable tel quel par URL.

Pour la maintenir à jour : éditer `public/openapi.yaml` à chaque table/colonne/policy ajoutée ou modifiée (même réflexe que pour [BACKEND.md](#), qui reste la source la plus détaillée — l'OpenAPI y renvoie pour le détail des politiques plutôt que de tout dupliquer).

Documentation associée

- [BACKEND.md](#) — schéma Supabase, politiques RLS, Storage, fonctions SQL, historique des correctifs. À tenir à jour à

chaque évolution du backend.